

Graph: Practice problems

1. Given an unweighted undirected graph, determine if it is connected. (Return true if there is a path between every pair of nodes. Otherwise, return false.)

```
function isConnected(Graph):
    // Graph is represented as an adjacency list
    n = number of vertices in Graph
    visited = empty set

    // Start DFS (or BFS) from the first node
    start = any node in Graph
    DFS(Graph, start, visited)

    // After traversal, if all nodes are visited, the graph is connected
    if size(visited) == n:
        return true
    else:
        return false
```

```
function DFS(Graph, node, visited):
    add node to visited
    for each neighbor in Graph[node]:
        if neighbor not in visited:
            DFS(Graph, neighbor, visited)
```

2. Given a graph, write pseudo code to determine the number of its components. (A component of a graph is a maximal connected subgraph. If the graph is connected, it has only one component)

```
function countComponents(Graph):
    n = number of vertices in Graph
    visited = empty set
    count = 0

    for each node in Graph:
        if node not in visited:
            // Found a new component
            count = count + 1
            DFS(Graph, node, visited)

    return count
```

```

function DFS(Graph, node, visited):
    add node to visited
    for each neighbor in Graph[node]:
        if neighbor not in visited:
            DFS(Graph, neighbor, visited)

```

3. You are given an $n \times n$ grid representing a room. Each cell in the grid can be either **open** or **blocked**:

- $\text{Room}[i][j] = 1$ means the cell is open (the robot can move there).
- $\text{Room}[i][j] = 0$ means the cell is blocked (the robot cannot move there).

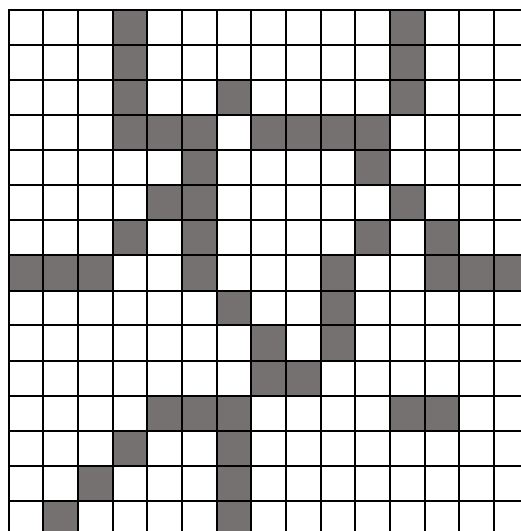
A robot is placed at a random position (s, t) in the grid. The robot can move **up**, **down**, **left**, or **right**, but only into adjacent cells that are open (i.e., have value 1).

There are four **exit doors** located at the **corners** of the grid: $(0, 0)$, $(0, n-1)$, $(n-1, 0)$, and $(n-1, n-1)$.

Your task is to design an **algorithm** that determines whether the robot can reach **at least one of the exit doors** from its starting position.

Write **pseudocode** for a function that takes the robot's starting position (s, t) as input and returns:

- `True` if the robot can reach any exit door
- `False` otherwise



We need to create a graph G where each node corresponds to a cell (i, j) in the grid. An edge exists between two nodes if the robot can move from one cell to another: the neighbors of $(i, j) = \{(i+1, j), (i-1, j), (i, j+1), (i, j-1)\}$.

```
function robot_exit(Room, s, t, n):
    createGraph(G)
    // G is an implicit graph based on Room; only cells with Room[i][j] == 1 are nodes
    start = node(s, t)
    DFS(G, start)
    // Check if any of the exit nodes were visited
    if isVisited(node(0, 0)) or isVisited(node(0, n-1)) or
       isVisited(node(n-1, 0)) or isVisited(node(n-1, n-1)):
        return true
    else:
        return false
```

Here is the python code:

```
def robot_exit(Room, s, t):
    n = len(Room)

    # Step 1: Create the graph
    G = {}
    for i in range(n):
        for j in range(n):
            if Room[i][j] == 1: # only open cells
                neighbors = []
                for (x, y) in [(i+1,j), (i-1,j), (i,j+1), (i,j-1)]:
                    if 0 <= x < n and 0 <= y < n and Room[x][y] == 1:
                        neighbors.append((x, y))
                G[(i, j)] = neighbors

    # Step 2: DFS to find reachable cells
    visited = set()

    def dfs(node):
        if node in visited:
            return
        visited.add(node)
        for neighbor in G.get(node, []):
            dfs(neighbor)

    start = (s, t)
    if Room[s][t] == 0:
        return False # start is blocked

    dfs(start)
```

```

# Step 3: Check if any exit corner is visited
exits = [(0, 0), (0, n-1), (n-1, 0), (n-1, n-1)]
for e in exits:
    if e in visited:
        return True
return False

```

4. You are given a directed graph where each node is painted either black or white. You are also given a starting node, s , which is white. Your task is to determine whether there exists a path in the graph that starts at s and ends at a different white node (not necessarily adjacent to s).

```

function hasPathToAnotherWhite(Graph, color, s):
    // Graph: adjacency list representation
    // color[node] = "white" or "black"
    visited = empty set

    function DFS(node):
        add node to visited
        for each neighbor in Graph[node]:
            if neighbor not in visited:
                if color[neighbor] == "white" and neighbor != s:
                    return true
                if DFS(neighbor):
                    return true
        return false

    return DFS(s)

```